Decision tree

Aymen Negadi

July 2026

1 Introduction

Decision Tree is one of the most popular supervised machine learning algorithms used for both classification and regression tasks. In this chapter, we focus on its application to classification problems.

Unlike Logistic Regression, which predicts probabilities using a mathematical function, a Decision Tree predicts the class of an observation by asking a sequence of questions about its features. Each question divides the dataset into smaller and more homogeneous subsets until a final decision is reached.

One of the greatest advantages of Decision Trees is their simplicity. The prediction process is intuitive and resembles human decision making, making the model easy to understand, visualize, and interpret.

Decision Trees are widely used in healthcare, finance, cybersecurity, marketing, and many other domains where explainable decisions are required.

2 What is a Decision Tree?

A Decision Tree is a tree-structured predictive model composed of decision rules. Starting from the root node, the algorithm evaluates a sequence of conditions until it reaches a final prediction.

Unlike mathematical models that rely on complex equations, Decision Trees represent knowledge as a collection of questions and answers.

For example, suppose a bank wants to determine whether a loan application should be approved.

The prediction process may follow the sequence

$$CreditScore \rightarrow Salary \rightarrow EmploymentStatus \rightarrow Decision$$

Each question partitions the data into smaller groups that become increasingly homogeneous.

3 Why is it Called a Decision Tree?

The algorithm receives its name because its graphical representation resembles an inverted tree.

A Decision Tree consists of

- one root node,
- several internal decision nodes,
- branches connecting the nodes,
- leaf nodes containing the final prediction.

Every prediction follows exactly one path from the root node to a leaf node.

4 Why Do We Use Decision Trees?

Many real-world decisions are naturally made through a sequence of questions.

For example, consider a university admission process.

Instead of using a single mathematical equation, the admission committee may evaluate

- Is the average score above 12?
- Does the student satisfy the language requirement?
- Is the required documentation complete?

Each answer determines the next question until a final decision is reached.

Decision Trees reproduce this decision-making process automatically by learning the optimal sequence of questions directly from the training data.

5 Binary Classification Example

Consider the following dataset.

The objective is to predict whether a future applicant will be approved.

Instead of estimating a probability, the Decision Tree successively asks questions about the applicant until a final class is assigned.

6 Components of a Decision Tree

A Decision Tree consists of four fundamental components.

6.1 Root Node

The root node is the first node of the tree.

It contains the complete training dataset and performs the first split.

Example

$$Salary > 50000?$$

6.2 Internal Nodes

Internal nodes contain decision rules.

Each internal node evaluates one feature and partitions the observations into smaller subsets.

Examples include

$$Age > 30?$$

or

$$CreditScore > 700?$$

6.3 Branches

Branches connect two nodes.

Each branch corresponds to one possible answer to a decision rule.

For example,

$$Salary > 50000?$$

creates two branches

Yes **No**

6.4 Leaf Nodes

Leaf nodes are the terminal nodes of the tree.

They contain the final prediction.

Examples include

$$Approved$$

or

$$Rejected$$

Once a leaf node is reached, the prediction process stops.

7 How a Decision Tree Works

The Decision Tree algorithm follows a recursive procedure.

Step 1

Begin with the complete training dataset.

Step 2

Search for the feature that best separates the classes.

Suppose the algorithm chooses

$$Salary > 50000?$$

Step 3

Split the dataset into two subsets.

Applicants satisfying the condition are sent to one branch.

The remaining applicants are sent to the other branch.

Step 4

Repeat exactly the same process for each subset.

If one subset still contains multiple classes, another decision rule is selected.

Step 5

Continue recursively until a stopping criterion is satisfied.

8 Recursive Splitting

Decision Trees use a recursive strategy.

The same sequence is repeated continuously.

$$ChooseBestFeature \rightarrow SplitDataset \rightarrow Repeat$$

Each split creates increasingly homogeneous groups.

Eventually every leaf node contains observations that mostly belong to a single class.

9 Real-World Applications

Decision Trees are used in numerous fields.

Healthcare

- Disease diagnosis
- Cancer prediction
- Medical treatment recommendation

Finance

- Credit scoring
- Loan approval
- Risk analysis

Cybersecurity

- Malware detection
- Intrusion detection
- Spam filtering

Marketing

- Customer segmentation
- Purchase prediction
- Churn prediction

Education

- Student performance prediction
- Graduation prediction

10 Advantages

Decision Trees provide several important advantages.

- Easy to understand.
- Easy to visualize.
- Highly interpretable.
- Requires little data preparation.
- Handles numerical and categorical features.
- Captures nonlinear relationships.
- Fast during prediction.

11 Limitations

Despite their advantages, Decision Trees also have several limitations.

- They may overfit the training data.
- Small variations in the dataset may produce different trees.
- Deep trees become difficult to interpret.
- Greedy splitting does not always produce the globally optimal tree.

12 Transition to Tree Construction

We now understand how Decision Trees make predictions.

However, one important question remains.

Suppose the dataset contains many features.

Why should the algorithm split using

$$Salary > 50000?$$

instead of

$$Age > 30?$$

or any other feature?

The answer is based on mathematical measures of impurity.

The next sections introduce the concepts of Entropy, Information Gain, and the Gini Index, which allow the algorithm to determine the best split at each node.

13 Impurity in Decision Trees

Before constructing a Decision Tree, it is necessary to determine the quality of each possible split. The objective is to partition the training data into subsets containing observations that belong predominantly to a single class.

The concept used to evaluate the quality of a split is called **impurity**.

A node is considered

- **Pure** if all observations belong to the same class.
- **Impure** if observations belong to different classes.

For example,

The goal of a Decision Tree is therefore to produce child nodes that are as pure as possible.

14 Entropy

Entropy is one of the most widely used measures of impurity in Decision Trees.

Originally introduced in Information Theory by Claude Shannon, entropy measures the amount of uncertainty or disorder in a dataset.

For Decision Trees,

- High entropy means high uncertainty.
- Low entropy means low uncertainty.
- Zero entropy indicates a perfectly pure node.

The entropy of a node is computed as

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

where

- S is the dataset,
- c is the number of classes,
- p_i is the probability of class i.

14.1 Special Cases

Case 1: Completely Pure Node

Suppose every observation belongs to the positive class.

$$P(Yes) = 1$$

$$P(No) = 0$$

Then

$$Entropy = -(1)\log_2(1) - (0)\log_2(0) = 0$$

Therefore,

$$Entropy = 0$$

A pure node contains no uncertainty.

Case 2: Maximum Uncertainty

Suppose the node contains

$$50\%$$

positive observations and

$$50\%$$

negative observations.
Then

$$Entropy = -0.5\log_2(0.5) - 0.5\log_2(0.5)$$

Since

$$\log_2(0.5) = -1,$$

we obtain

$$Entropy = 1.$$

This is the maximum possible entropy for binary classification.

14.2 Numerical Example

Suppose a node contains

- 8 Approved loans
- 2 Rejected loans

Then

$$P(Approved) = 0.8$$

$$P(Rejected) = 0.2$$

The entropy becomes

$$Entropy = -0.8\log_2(0.8) - 0.2\log_2(0.2)$$

Using

$$\log_2(0.8) \approx -0.322$$

and

$$\log_2(0.2) \approx -2.322,$$

we obtain

$$Entropy = -(0.8)(-0.322) - (0.2)(-2.322)$$

$$Entropy = 0.258 + 0.464 = 0.722.$$

This value indicates that the node still contains some uncertainty.

15 Information Gain

Entropy tells us how impure a node is.

However, the Decision Tree must decide

which feature produces the best split.

To answer this question, the algorithm computes the **Information Gain**.

Information Gain measures how much impurity decreases after splitting a node.

The formula is

$$IG = Entropy(Parent) - \sum_{i=1}^{k} \frac{|S_i|}{|S|} Entropy(S_i)$$

where

- S is the parent dataset,
- S_i are the child datasets,
- $|S_i|$ is the number of observations in child i.

A larger Information Gain indicates a better split.

15.1 Example

Suppose

Parent entropy

$$0.95$$

After splitting,

Left child entropy

$$0.20$$

Right child entropy

$$0.30$$

Each child contains half of the observations.

Therefore,

$$IG = 0.95 - (0.5 \times 0.20 + 0.5 \times 0.30)$$

$$= 0.95 - 0.25 = 0.70.$$

The split reduces uncertainty significantly.

16 Gini Index

Another popular impurity measure is the Gini Index.

Unlike entropy, it does not use logarithms.

The Gini Index is defined as

$$Gini = 1 - \sum_{i=1}^{c} p_i^2.$$

For binary classification,

$$Gini = 1 - (p_1^2 + p_2^2).$$

16.1 Example

Suppose

$$P(Yes) = 0.8,$$

$$P(No) = 0.2.$$

Then

$$Gini = 1 - (0.8^2 + 0.2^2)$$

$$= 1 - (0.64 + 0.04)$$

$$= 0.32.$$

A smaller Gini value indicates a purer node.

17 Entropy vs. Gini Index

Both measures usually produce very similar Decision Trees.

18 Choosing the Best Split

At every node, the algorithm evaluates all possible features.

For each candidate split, it computes

- Entropy or Gini Index,
- Information Gain (or Gini Reduction).

The feature producing the largest reduction in impurity is selected.

The process repeats recursively until one of the stopping criteria is met.

19 Prediction Process

After training, classification is straightforward.

Given a new observation,

1. Start at the root node.
2. Evaluate the decision rule.
3. Follow the corresponding branch.
4. Repeat until a leaf node is reached.
5. Output the class stored in the leaf.

Unlike Logistic Regression, Decision Trees do not compute probabilities using a mathematical function during prediction. Instead, predictions are obtained by traversing the learned tree according to the feature values of the new observation.

Table 1: Loan approval dataset

Age	Salary	Loan Approved
22	30000	No
35	60000	Yes
42	70000	Yes
24	25000	No
31	55000	Yes

Table 2: Examples of node purity

Node	Purity
Yes, Yes, Yes, Yes	Pure
No, No, No	Pure
Yes, No, Yes, No	Impure
Yes, Yes, No, Yes	Slightly Impure

Table 3: Comparison of Entropy and Gini Index

Criterion	Entropy	Gini Index
Formula	Uses logarithms	Uses squared probabilities
Range	0 to 1	0 to 0.5 (binary)
Computation	Slightly slower	Faster
Common Algorithm	ID3, C4.5	CART